

-Proiect-

Proiectarea Sistemelor Numerice

AUTOMAT BANCAR



Autori: Davidescu Lucian
Ichim Daniel Alexandru
Grupa: 30212

Profesor îndrumător: Prof Dr.Ing Octavian Creț

CUPRINS

Specificția proiectului.....	3
Schema bloc cu componente principale.....	4
Componente	5
Identificator de card.....	6
Memoria ROM	7
Memoria RAM	9
Demultiplexor	11
Cititor de PIN	13
Cititor de sumă	16
Afișor BCD7.....	18
Comparatoare.....	20
Scăzător	22
Numărător bancnote.....	23
Eliberare chitanță.....	24
Justificarea soluției alese.....	26
Instrucțiuni de utilizare	27
Posibilități de dezvoltare ulterioară	30

1.Specificația proiectului

Să se proiecteze un automat bancar care permite extrageri de sume în EURO. Se presupune că suma maximă care poate fi extrasă o dată este de maximum 1.000 EURO. Operația efectuată inițial este de identificare a cardului. Dacă această operație se încheie cu succes urmează alegerea operației de efectuat în continuare și anume interogare sold sau retragere numerar. În cazul cererii de interogare sold se afișează suma. Apoi se poate face alegerea de retragere numerar sau se eliberează cardul și, eventual, chitanța. În cazul cererii de eliberare de numerar se introduce suma, se verifică existența sumei cerute și se actualizează contul. Apoi se eliberează cardul, suma și, eventual, chitanța.

Proiectul este implementat în totalitate în VHDL. Testarea și funcționalitatea acestuia sunt prezentate cu ajutorul simulatorului Active-HDL și încărcare pe plăcuța de FPGA. Proiectul își propune realizarea unui automat bancar, care să simuleze două dintre opțiunile efectuate de către un bancomat, retragere numerar și interogare sold.

2.Schema bloc cu **componentele principale**

Avem intrările:

- ADRESA_CARD_PIN_SUMA - Vectorul de 5 biți pe care se vor citi pe rând, adresa cardului, pin-ul si suma introdusa de utilizator;
- BUT,BUT1 – pentru sincronizarea componentelor;
- RESET – pentru resetarea componentelor
- CLK_AFISOR – pentru afisorul BCD7 segmente;
- SEL_PIN – selecție biți la introducerea PIN-ului;
- SEL_SUMA_DEMUX_CHITANTA – selecție biți la introducerea sumei, alegerea opțiunii pentru demultiplexor, și opțiunii pentru chitanță.
- CLR – resetare afișor BCD7 segmente;

Componentele principale:

- Identificator de card
- Memorie ROM în care sunt reținute codurile PIN pentru toate conturile bancare;
- Afișor BCD7 segmente pentru afișarea soldului clientului in hexazecimal;
- Demultiplexor pentru alegerea operației;
- Memorie RAM în care sunt reținute soldurile;
- Cititor PIN care verifică validitatea PIN-ului cu ajutorul unor comparatoare;
- Cititor de sumă care verifică validitatea sume introduse și un numărător de bancnote care verifica disponibilitatea de bancnote în bancomat pentru eliberarea sumei solicitate de client.

3.Componente

1.Identificator de card (IDEN_CARD)

- Transformă vectorul de 5 biți "ADRESA" introdus de utilizator într-un întreg care va fi adresa din memoriile RAM (în care este reținut soldul clientului) și ROM (în care este reținut codul PIN al clientului).

-Codul:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity IDEN_CARD is
    port(ADRESA_CARD: in std_logic_vector(4 downto 0);
        BUT: in std_logic; --Buton
        ENAB: in std_logic ;
        RESET: in std_logic;
        ADRESA: inout INTEGER range 0 to 31 ;
        IES: out std_logic);
end IDEN_CARD;

architecture ARCH_IDEN_CARD of IDEN_CARD is
begin
    process (BUT, RESET)
    begin
        if (BUT='1' and ENAB='1') then
            case ADRESA_CARD is
                when "00001" => ADRESA <=1;
                when "00010" => ADRESA <=2;
                when "00011" => ADRESA <=3;
                when "00100" => ADRESA <=4;
                when "00101" => ADRESA <=5;
                when "00110" => ADRESA <=6;
                when "00111" => ADRESA <=7;
                when "01000" => ADRESA <=8;
                when "01001" => ADRESA <=9;
                when "01010" => ADRESA <=10;
                when "01011" => ADRESA <=11;
                when "01100" => ADRESA <=12;
                when "01101" => ADRESA <=13;
                when "01110" => ADRESA <=14;
                when "01111" => ADRESA <=15;
                when others => ADRESA <=16;
            end case;
        end if;
        if (ADRESA>0 and ADRESA<16) then IES<='1';
        else IES<='0';
    end process;
end ARCH_IDEN_CARD;
```

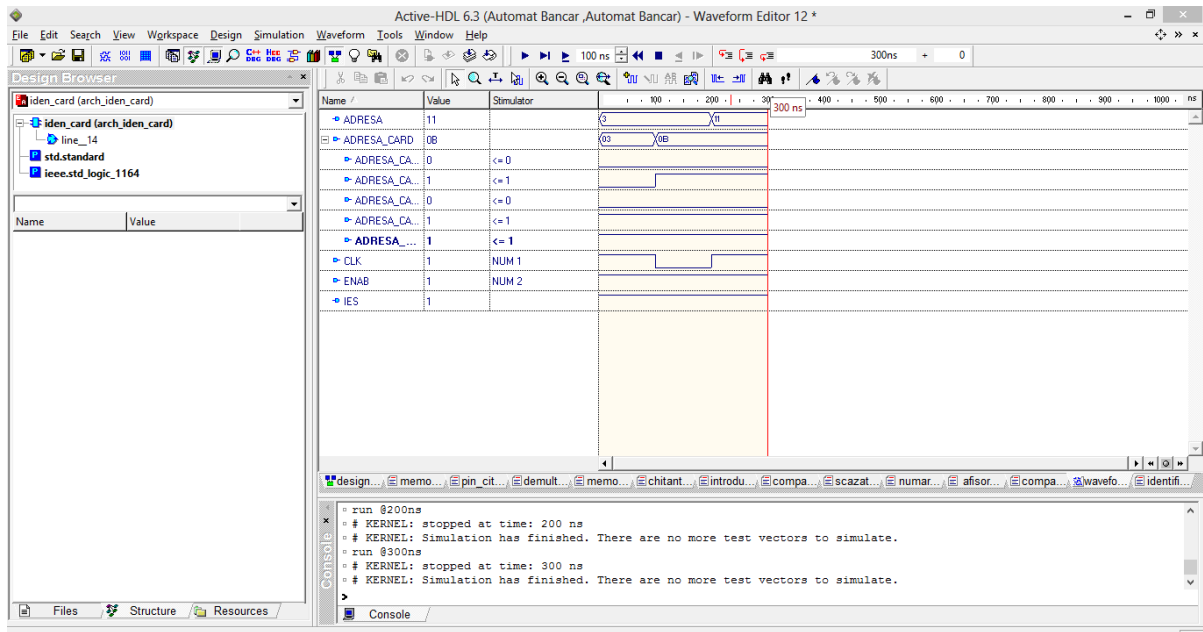
```

end if;
if (RESET='1') then ADRESA<=0; IES<='0';
end if;
end process;

```

end ARCH_IDEN_CARD;

-Exemplificare waveform - IDENTIFICATOR DE CARD:



2. Memoria ROM (MEMORIE_ROM_PINURI)

- În memoria ROM sunt reținute codurile PIN ale clienților. Pe intrarea “adresa_pin” de tip întreg se introduce adresa din memorie în care este reținut codul PIN, iar în vectorul de 16 biți se va încărca conținutul memoriei.

-Codul:

```

library ieee;
use ieee.std_logic_1164.all;

entity pin is
    port (adresa_pin: in integer range 0 to 31 ; -- 32 de
pinuri, 5 biti pentru adrese.
        memorie_pin:out std_logic_vector(15 downto 0)); --pinul
are 4 cifre ,deci 16 biti pentru pin.
end pin;

architecture arch_pin of pin is
type mm is array (31 downto 0) of std_logic_vector(15 downto
0);

```

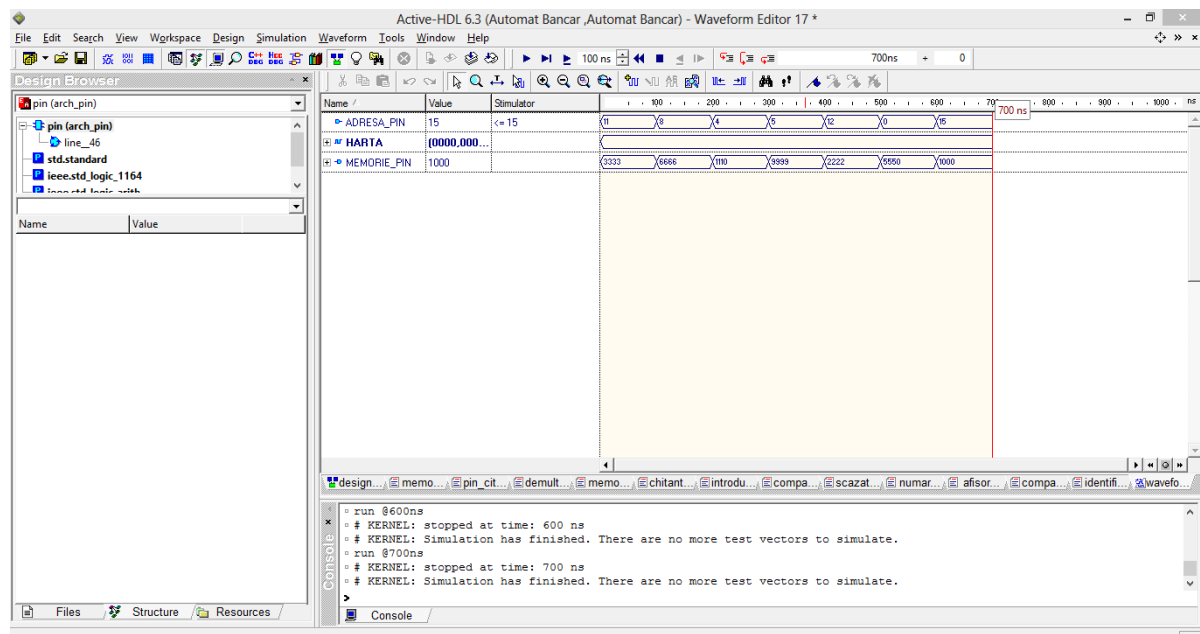
```

signal harta: mm:=("0000000000000000", --0000 31
                   "0000000000000000", --0000 30
                   "0000000000000000", --0000 29
                   "0000000000000000", --0000 28
                   "0000000000000000", --0000 27
                   "0000000000000000", --0000 26
                   "0000000000000000", --0000 25
                   "0000000000000000", --0000 24
                   "0000000000000000", --0000 23
                   "0000000000000000", --0000 22
                   "0000000000000000", --0000 21
                   "0000000000000000", --0000 20
                   "0000000000000000", --0000 19
                   "0000000000000000", --0000 18
                   "0000000000000000", --0000 17
                   "0000000000000000", --0000 16
                   "0001000000000000", --1000 15
                   "0000000010011001", --0099 14
                   "0001000100010001", --1111 13
                   "0010001000100010", --2222 12
                   "0011001100110011", --3333 11
                   "0100010001000100", --4444 10
                   "0101010101010101", --5555 9
                   "0110011001100110", --6666 8
                   "0111011101110111", --7777 7
                   "1000100010001000", --8888 6
                   "1001100110011001", --9999 5
                   "0001000100010000", --1110 4
                   "0010001000100000", --2220 3
                   "0011001100110000", --3330 2
                   "0100010001000000", --4440 1
                   "0101010101010000"); --5550 0

begin
    memorie_pin<=harta(adresa_pin);
end arch_pin;

```

-Exemplificare waveform - memorie ROM:



3. Memoria RAM (MEMORIE_RAM_SOLDURI)

- În memoria RAM sunt reținute soldurile clienților . Pe intrarea “adresa_card” de tip întreg se transmite adresa din memorie la care este reținut soldul clientului, pe intrarea SUMA se transmite soldul curent al clientului, în caz că acesta a efectuat operația de retragere numerar suma din memoria RAM va trebui modificată, intrarea WE activează modul de scriere în memoria RAM în funcție de opțiunea aleasă de client, astfel se înlocuiește suma de la adresa respectivă.

-Codul:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity MEMORIE_SOLD is
    port(ADRESA_CARD: in INTEGER range 0 to 31;
         WE: in std_logic; -- WE: Write Enable, la activare se
         inlocuieste suma de la adresa cardului cu SUMA
         SUMA: in std_logic_vector(15 downto 0);
         SOLD_CLIENT: out std_logic_vector(15 downto 0));
end MEMORIE_SOLD;

architecture ARCH_MEMORIE_SOLD of MEMORIE_SOLD is
    type MM is array(31 downto 0) of std_logic_vector(15 downto 0);
    process (WE, ADRESA_CARD)
    variable HARTA: MM:=("0000000000000000", --0      31
                          "0000000000000000", --0      30
                          "0000000000000000", --0      29
                          "0000000000000000", --0      28
                          "0000000000000000", --0      27
                          "0000000000000000", --0      26
                          "0000000000000000", --0      25
                          "0000000000000000", --0      24
                          "0000000000000000", --0      23
                          "0000000000000000", --0      22
                          "0000000000000000", --0      21
                          "0000000000000000", --0      20
                          "0000000000000000", --0      19
                          "0000000000000000", --0      18
                          "0000000000000000", --0      17
                          "0000000000000000", --0      16
                          "0000000011110000", --240     15
```

```

"0000000101110010", --370 14
"0000000110100100", --420 13
"0000001000100110", --550 12
"0000111100001010", --3850 11
"0000100001100110", --2150 10
"0000001010001010", --650 9
"0000000001111000", --120 8
"0000000101101000", --360 7
"0000001000110000", --560 6
"0000001101111010", --890 5
"0000001001110110", --630 4
"0000000001101110", --110 3
"0000000001011010", --90 2
"0000000011010010", --210 1
"0000000000000000"); --1111 0

```

begin

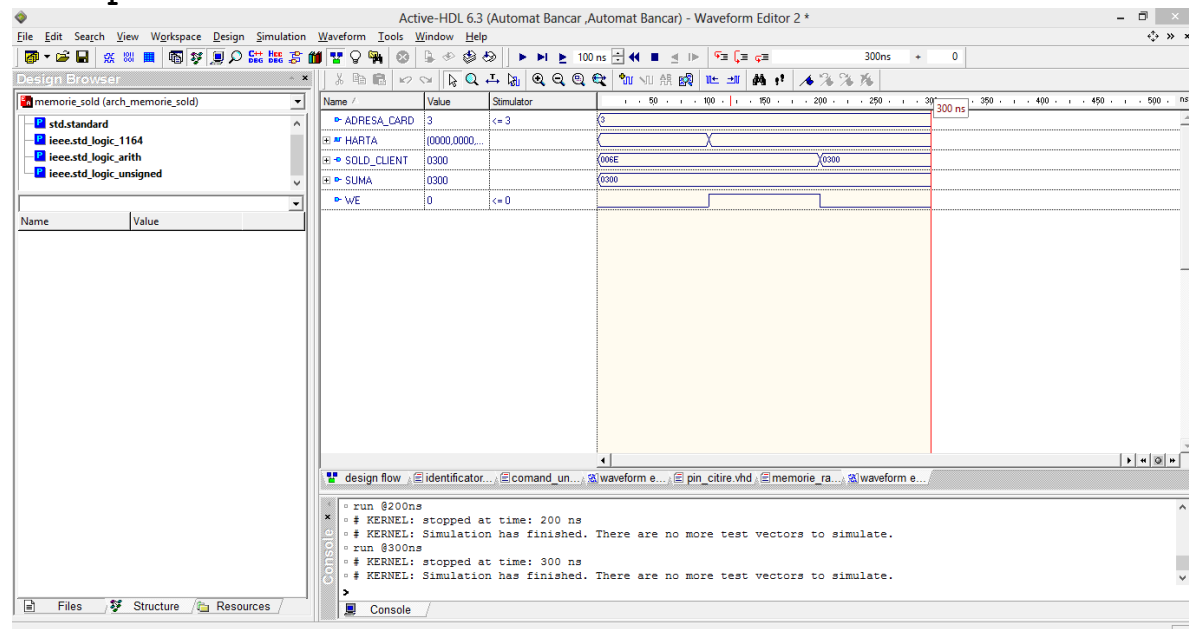
```

    if WE='1' then
        HARTA(ADRESA_CARD) := SUMA ;
    else SOLD_CLIENT <= HARTA(ADRESA_CARD) ;
    end if;
end process;

```

end ARCH_MEMORIE_SOLD;

-Exemplificare waveform - MEMORIE RAM:



4. Demultiplexor (DEMULTIPLEXOR) – pentru alegerea operației

-Cu ajutorul demultiplexorului se efectuează alegerea operației efectuate de client, acesta primește intrarea de date de la cititorul de PIN, în cazul în care PIN-ul introdus de utilizator este valid, se activează demultiplexorul având intrarea generală de selecție S.

- În cazul în care pe 'S' avem:

- valoare logică '1' se va activa ieșirea pentru interogare sold (afișare sold);

-valoare logică '0' se va active ieșire pentru retragere numerar;

-Demultiplexorul este activat doar în cazul în care PIN-ul introdus de utilizator este corect.

-Codul:

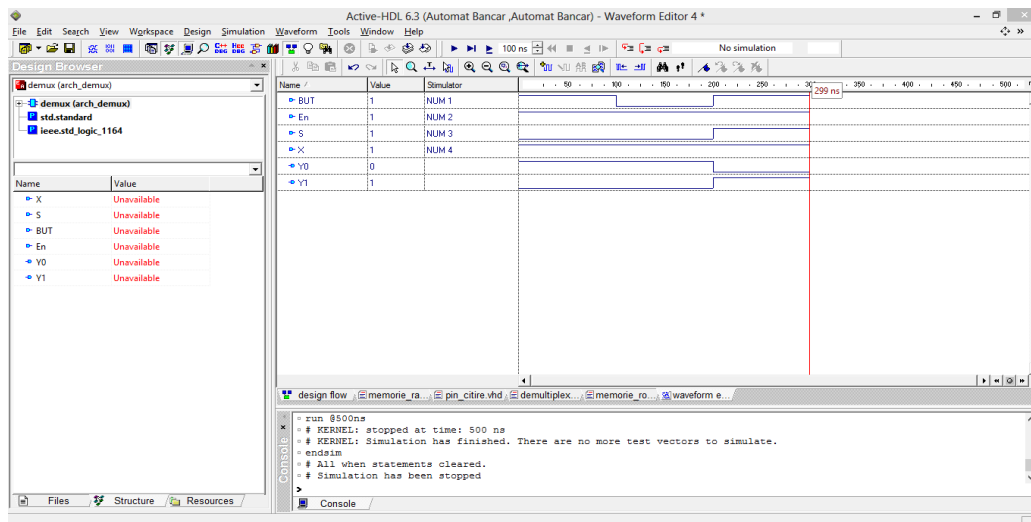
```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity DEMUX is
    port(
        RESET: in std_logic ;
        S: in std_logic;      --S: selectia
        CLK: in std_logic;
        Enab: in std_logic;
        Y0,Y1: out std_logic ); --Y: Iesirea
end DEMUX;

architecture ARCH_DEMUX of DEMUX is
begin
    process (CLK,RESET)
    begin
        if (CLK='1' and Enab='1') then
            if S='0' and CLK='1' then Y0<='1'; Y1<='0'; --
Retragere Numerar
            elsif S='1' and CLK='1' then Y1<='1'; Y0<='0';--
Afisare Sold
            end if;
            elsif RESET='1' then
                Y1<='0'; Y0<='0';
            end if;
        end process;

end ARCH_DEMUX;
```

-Exemplificare waveform DEMULTIPLEXOR:



5. Cititor de PIN – (PIN_CITIRE)

-Cu ajutorul cititorului de PIN se verifică validitatea PIN-ului introdus de către utilizator. Pe intrarea “ADRESA_CARD” se transmite un întreg care reprezintă adresa contului din memoria ROM respectiv RAM, cu ajutorul intrării “SEL” se citesc pe rând câte 4 biți memorați pe vectorul “x”, în total PIN-ul va fi alcătuit din 16 biți. Cu ajutorul unui comparator se verifică validitatea PIN-ului, comparându-l cu cel citit din memoria ROM.

-Intrarea de “enable” activează această unitate, în cazul în care adresa de cont a clientului este una validă. Intrarea de disable, dezactivează această unitate, dacă clientul a introdus un PIN corect.

-Ieșirea COUT_PIN se activează la introducerea greșită de 3 ori consecutive a PIN-ului, astfel se aprinde un LED reprezentând faptul că a fost reținut cardul. Ultima variabilă de ieșire, “IESIRE” va avea valoarea ‘1’ dacă PIN-ul introdus de utilizator este corect altfel va fi ‘0’, aceasta urmând să fie intrare de enable pentru demultiplexorul de alegere a opțiunii.

-Codul:

```
library IEEE;  
use IEEE.std_logic_1164.all;
```

```
entity citire_pin is  
  port(  
    ADRESA_CARD: in INTEGER range 0 to 31;  
    SEL_PIN: in std_logic_vector(1 downto 0); --selectii
```

```

    X_PIN:in std_logic_vector(4 downto 0);--valoare introdusa
de la switch
    ENABLE: in std_logic;
    BUT:in std_logic;
    RESET:in std_logic;
    COUT_PIN: out std_logic; --se activeaza la introducerea
gresita de 3 ori a pinului
    IESIRE: out std_logic; --se activeaza daca este pinul
gresi
    IESIRE2: out std_logic
);
end citire_pin;

architecture arh_citire_pin of citire_pin is

component PIN
    PORT (ADRESA_PIN: in integer range 0 to 31 ; -- 32 de
pinuri, 5 BITI pentru adrese.
    MEMORIE_PIN:OUT STD_LOGIC_VECTOR(15 DOWNT0 0)); --pinul
are 4 cifre ,deci 16 biti pentru pin.
end component;

component COMP_EGAL IS
    PORT (
    A,B:IN STD_LOGIC_VECTOR(15 DOWNT0 0) ;
    BUT: in std_logic;
    C:OUT STD_LOGIC);
end component;

component PIN_GRESIT
    PORT (
    A: in std_logic;
    COUT_PIN: out std_logic;
    reset_pin: out std_logic;
    PIN_GRESIT: out std_logic
    );
end component;

signal PIN2: std_logic_vector(15 downto 0);--Pinul din memorie
signal pin1: std_logic_vector(15 downto 0):="-----
";--Pinul introdus de utilizator
signal IESIRE1: std_logic;
signal reset_pin:std_logic;

begin
    C1:process (BUT,RESET)
    begin
        if BUT='1' and ENABLE='1'then
            case SEL_PIN is
                when "00"=>pin1(3 downto 0)<=X_PIN(3 downto 0);
                when "01"=>pin1(7 downto 4)<=X_PIN(3 downto 0);

```

```

        when "10"=>pin1(11 downto 8)<=X_PIN(3 downto 0);
        when "11"=>pin1(15 downto 12)<=X_PIN(3 downto 0);
        when others => pin1(15 downto 0) <= (others =>
'U'));
        end case;
    end if;
    if RESET='1' or reset_pin='1' then
        pin1<="-----";

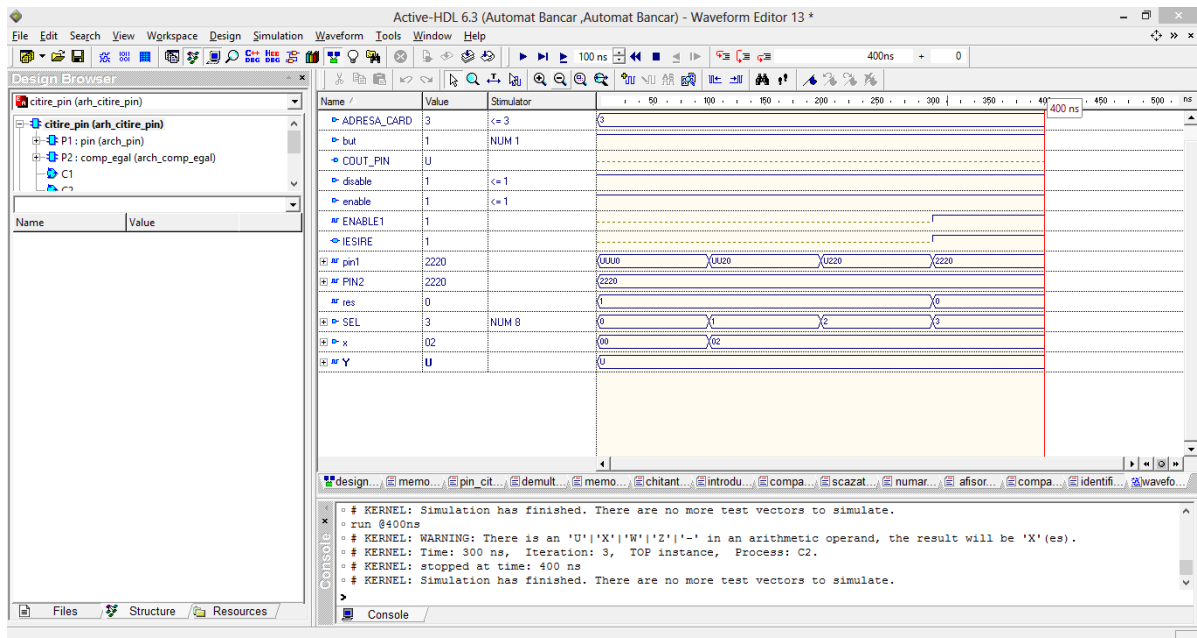
    end if;
end process;

P1:PIN port map(ADRESA_CARD,PIN2);
P2:COMP_EGAL port map(PIN2,pin1,BUT,IESIRE1);
P3:PIN_GRESIT port map(IESIRE1,COUT_PIN,reset_pin,IESIRE);

IESIRE2<=IESIRE1 when IESIRE1='1';
end arh_citire_pin;

```

Exemplificare waveform - CITITOR DE PIN:



6. Cititor de suma (INTRODUCERE_SUMA)

-Cu ajutorul acestei componente, se citește suma introdusă de utilizator, fiind reprezentată de un vector pe 10 biți, se citesc pe rând câte 5 biți, suma maximă admisă fiind de 1000 de euro. Pentru această entitate avem următoarele intrări, intrarea de enable responsabilă pentru activarea acestei componente, provenind de la demultiplexorul pentru alegerea opțiunii, fiind activă doar în cazul alegerii de retragere numerar. Intrarea de "SEL" este folosită pentru citirea a 10 biți, mai întâi primii 5 iar apoi ultimii 5 dacă mai este nevoie, biți care se citesc de pe vectorul "X_SUMA". Pe intrarea "SOLD_CLIENT" se transmite soldul curent al clientului, provenit din memoria RAM sub formă de întreg.

-Avem ieșirile, "iesire" care se activează doar în cazul în care suma introdusă de utilizator este validă, verificare care se realizează cu ajutorul unor comparatoare, suma trebuie să fie mai mică sau egală cu 1000 și cu soldul clientului din memoria RAM. Pe variabila "SUMA_INT" se transmite suma introdusă de utilizator sub formă de întreg.

-Codul:

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity INTR_SUMA is
    port(
        enable:in std_logic;
        SEL:in std_logic;        --Selectii pentru introducere
        cifrelor(0-primele 5 cifre, 1-urmatoarele 5 cifre daca mai
        trebuie
        X_SUMA :in std_logic_vector(4 downto 0); --Introducerea
        sumei
        BUT: in std_logic;        --Button sau Clk
        SOLD_CLIENT: in integer range 0 to 9999;
        RESET: in std_logic;
        iesire: out std_logic ;   --Se activeaza daca totul este in
        regula,adica <1000 si <decat sold-ul clientului;
        SUMA_INT: out integer range 0 to 1000); --Suma introdusa
end INTR_SUMA;

architecture ARCH_INTR_SUMA of INTR_SUMA is
```

```

signal suma_introdusa: std_logic_vector(9 downto 0) := "-----
--";
signal ENABLE1: std_logic:='0'; --Enable pentru comparatorul
mai mic ca 1000
signal IES: std_logic:='L'; --Iesirea de la comparatorul mai
mic ca 1000,1-Daca este , 0-Daca nu
signal SUMA_INTRODUSA_REZ: integer range 0 to 9999; --Suma
convertita in integer
signal IES1:std_logic:='L'; --IESIRE de la comparatorul mai
mic ca sold-ul clientului,1-Daca este,0-Daca nu
signal iesire1: std_logic;

component COMP
    PORT (A:IN INTEGER range 0 to 9999;
        BUT: in std_Logic;
        SUMA: in std_logic_vector(9 downto 0);
        C:OUT STD_LOGIC); --IESIREA VA AVEA VALOAREA 1 DACA
B<=A SI 0 IN CAZ CONTRAR
end component;
begin
    process (BUT,RESET)
        variable PLUS: integer range 0 to 2:=0;
    begin
        if BUT='1' and enable='1' then
            case sel is
                when '0'=>suma_introdusa(4 downto 0)<=X_SUMA;
                when '1'=>suma_introdusa(9 downto 5)<=X_SUMA;
                when others => suma_introdusa <= (others => '0');
            end case;
        end if;
        if RESET='1' or IES='L' or IES1='L' then
            suma_introdusa<="-----";

        end if;
    end process;

    D1:COMP port map(1000,BUT,suma_introdusa,IES);
    D2:COMP port map(SOLD_CLIENT,BUT,suma_introdusa,IES1);

    iesire<=IES and IES1;
    iesire1<=IES AND IES1;
    SUMA_INT<=CONV_INTEGER(suma_introdusa) when iesire1='1';

end ARCH_INTR_SUMA;

```


7. Afișor BCD7 segmente (AFISOR)

-Cu ajutorul acestei componente, se realizează afișarea sumei din contul clientului pe afișoarele BCD7, în cazul în care acesta a selectat opțiunea de interogare sold.

-Codul:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_UNSIGNED.all;

entity x7seg is
    port(
        enable: in std_logic;
        clk: in STD_LOGIC;
        clr: in STD_LOGIC;
        a_to_g: out STD_LOGIC_VECTOR(6 downto 0);
        an: out STD_LOGIC_VECTOR(3 downto 0);
        x: in STD_LOGIC_VECTOR(15 downto 0)
    );
end x7seg;

architecture arh_x7seg of x7seg is
    signal s: STD_LOGIC_VECTOR(1 downto 0);
    signal digit: STD_LOGIC_VECTOR(3 downto 0);
    signal clkdiv: STD_LOGIC_VECTOR(18 downto 0);

begin

    s<=clkdiv(18 downto 17);

    process(s, x, enable)
    begin
        CASE s IS
            WHEN "00" => digit <= x(3 downto 0);
            WHEN "01" => digit <= x(7 downto 4);
            WHEN "10" => digit <= x(11 downto 8);
            WHEN OTHERS => digit <= x(15 downto 12);
        END CASE;
    end process;

    -- Decodificare
    process(digit)
    begin
        CASE digit is
            when "0000"=> a_to_g <="0000001";  -- '0'
```

```

when "0000"=> a_to_g <="1000000"; -- '0'
when "0001"=> a_to_g <="1111001"; -- '1'
when "0010"=> a_to_g <="0100100"; -- '2'
when "0011"=> a_to_g <="0110000"; -- '3'
when "0100"=> a_to_g <="0011011"; -- '4'
when "0101"=> a_to_g <="0010010"; -- '5'
when "0110"=> a_to_g <="0000010"; -- '6'
when "0111"=> a_to_g <="1111000"; -- '7'
when "1000"=> a_to_g <="0000000"; -- '8'
when "1001"=> a_to_g <="0010000"; -- '9'
when "1010"=> a_to_g <="0001000"; -- 'A'
when "1011"=> a_to_g <="0000011"; -- 'b'
when "1100"=> a_to_g <="1000110"; -- 'C'
when "1101"=> a_to_g <="0100001"; -- 'd'
when "1110"=> a_to_g <="0000110"; -- 'E'
when "1111"=> a_to_g <="0001110"; -- 'F'
when others=> a_to_g <="1111111";

END CASE;
end process;

-- Selectare Anod
process(s)
begin
    an <= "1111";
    if clr = '0' then
        an(conv_integer(s)) <= '0';
    end if;
end process;

--end process;
-- Divizor ceas
process(clk, clr)
begin
    if clr = '1' and enable='1' then
        clkdiv <= (others => '0');

        elsif clk'event and clk = '1' and enable='1' then
            clkdiv <= clkdiv + 1;
        end if;
    end process;
end arh_x7seg;

```

8. Comparatoare (COMP_INTRODUCERE_PIN/ COMP_INTRODUCERE_SUMA)

-Comparatoarele au un rol important în acest proiect, ele verifică validitatea datelor introduse de client. Primul COMPARATOR verifică dacă o variabilă 'a' este mai mică sau egală decât o variabilă 'b', acesta fiind utilizat la compararea sumei introduse cu 1000 și totodată cu soldul clientului. Cel de-al doilea, COMPARATOR_CARD_PIN, verifică dacă o variabilă 'a' este egală cu o variabilă 'b', acesta fiind utilizat pentru compararea PIN-ului introdus de client cu PIN-ul aflat în memoria ROM. Ultimul comparator, COMPARATOR_CARD_EXISTA, verifică dacă CARD-ul introdus de client este valid și se află în memoria bancomatului.

-Codul:

(COMP_INTRODUCERE_PIN) – COD:

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY COMP_EGAL IS
    PORT (
        A,B:IN STD_LOGIC_VECTOR(15 DOWNTO 0) ;
        BUT: in std_logic;
        C:OUT STD_LOGIC) ;
END COMP_EGAL;

ARCHITECTURE ARCH_COMP_EGAL OF COMP_EGAL IS
BEGIN
    PROCESS (A,B,BUT)
        variable INTER: std_logic;

    BEGIN
        if BUT='1' then
            if B(3 downto 0) = "----" or B(7 downto 4) = "----"
or B(11 downto 8) = "----" or B(15 downto 12) = "----" then
                INTER:='0'; C<='0';
            else INTER:='1';
            end IF;
            if(INTER = '1') then
                IF A=B then C<='1';
                else C<='L';
                end if;
            end if;
        end if;
    end process;
```

```

        END IF;

    END PROCESS;
END ARCH_COMP_EGAL;

(COMP_INTRODUCERE_SUMA)- COD:
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY COMP IS
    PORT (A:IN INTEGER range 0 to 9999;
          BUT: in std_logic;
          SUMA: in std_logic_vector(9 downto 0);
          C:OUT STD_LOGIC); --IESIREA VA AVEA VALOAREA 1 DACA
B<=A SI 0 IN CAZ CONTRAR
END COMP;

ARCHITECTURE ARCH_COMP_MAI_MIC_SAU_EGAL OF COMP IS
BEGIN
    PROCESS(SUMA,BUT)
        variable INTER: std_logic;
        variable SUMA_INTRODUSA_REZ: integer range 0 to 9999;
    BEGIN
        if BUT='1' then
            IF SUMA(9 downto 5)="-----" or SUMA(4 downto 0)="-
-----" THEN
                INTER:='0';    C<='0';
            else INTER:='1';
SUMA_INTRODUSA_REZ:=(CONV_INTEGER(SUMA));
                end if;

                if INTER='1' then
                    if(SUMA_INTRODUSA_REZ<A) then C<='1';
                    else C<='L';
                    end if;
                end if;
            END IF;
        END PROCESS;
END ARCH_COMP_MAI_MIC_SAU_EGAL;

```

9.Scăzător (SCAZATOR) – pentru alegerea operației

-Este utilizat pentru a realiza, diferența dintre soldul clientului și suma introdusă de acesta în cazul retragerii de numerar, pentru a realiza actualizarea sumei din cont, după încheierea efectuării operației de către client.

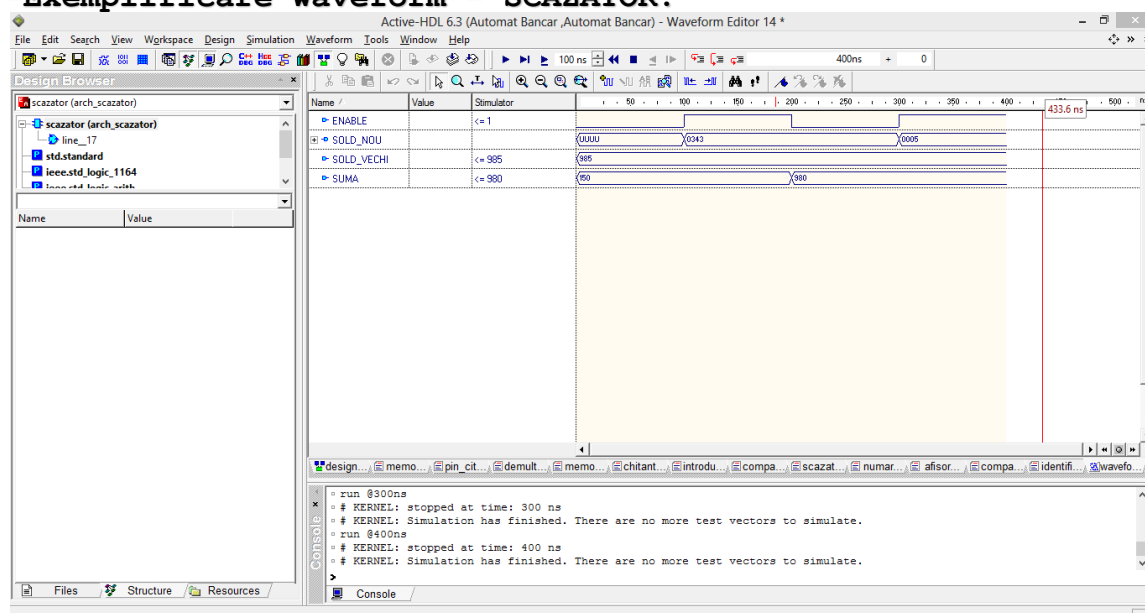
-Codul:

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity SCAZATOR is
    port(
        SOLD_VECHI: in integer range 0 to 10000;
        SUMA: in integer range 0 to 1000;
        SOLD_NOU: out std_logic_vector(15 downto 0);
        ENABLE: in std_logic);
end SCAZATOR;

architecture ARCH_SCAZATOR of SCAZATOR is
begin
    process(ENABLE)
    begin
        if(ENABLE'EVENT and ENABLE='1') then
            SOLD_NOU <= STD_LOGIC_VECTOR(to_unsigned(SOLD_VECHI -
SUMA,16)) ;
        end if;
    end process;
end ARCH_SCAZATOR;
```

-Exemplificare waveform - SCAZATOR:



10. Numărător bancnote (NUMARATOR_BANCNOTE)

-Această entitate este responsabilă pentru contabilizarea numărului necesar de bancnote pentru eliberarea sumei dorite de către client. De asemenea este posibil ca bancomatul să nu aibă numărul necesar de bancnote pentru eliberarea sumei, astfel se activează ieșirea "CIN". Pe intrarea "A" se transmite suma introdusă de utilizator sub formă de întreg, iar ieșirea "Y" se activează dacă s-au eliberat cu succes bancnotele.

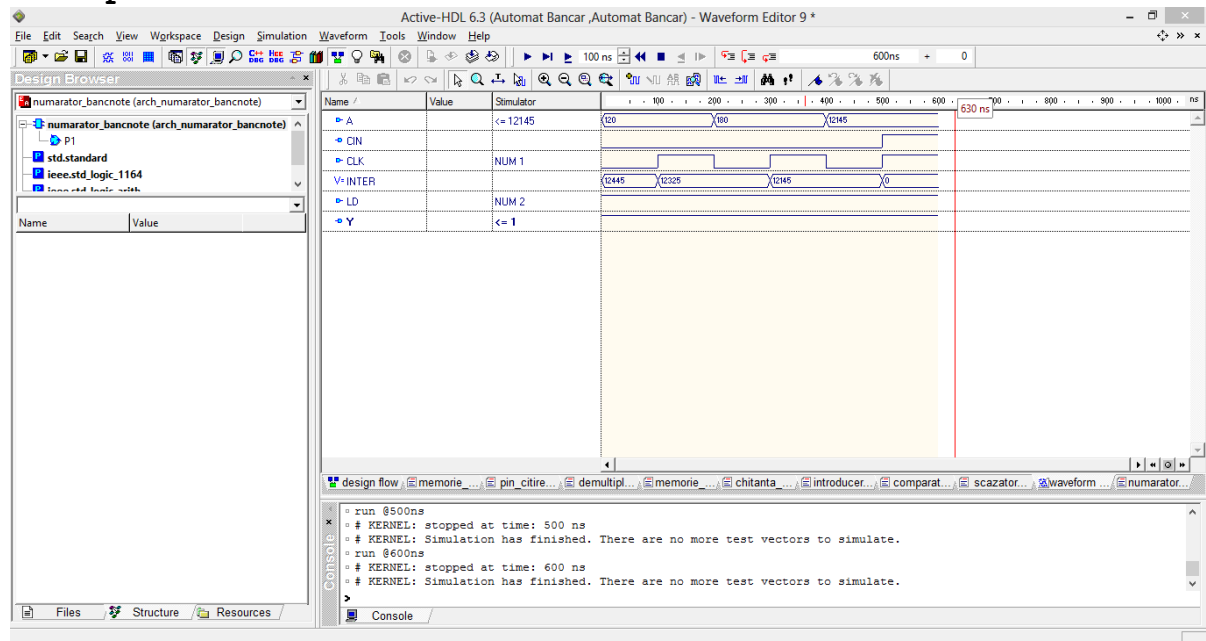
-Codul:

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY NUMARATOR_BANCNOTE IS
    PORT(LD,CLK:IN STD_LOGIC;
         A:IN integer range 0 to 100000;
         Y:out std_logic;           --Daca s-au eliberat bacnotele
         CIN:OUT STD_LOGIC);       -- CAND NU MAI SUNT BANCNOTE CIN
DEVINE 1
END NUMARATOR_BANCNOTE;

ARCHITECTURE ARCH_NUMARATOR_BANCNOTE OF NUMARATOR_BANCNOTE IS
BEGIN
    P1:PROCESS (CLK)
        VARIABLE INTER:integer:=12445; --12445 bacnote de un euro
    BEGIN
        IF CLK='1' AND CLK'EVENT AND LD='1' THEN
            INTER:=INTER-A;
        END IF;
        IF INTER<1 and LD='1' THEN CIN<='1';
            elsif LD='1' then Y<='1';
        END IF;
    END PROCESS;
END ARCH_NUMARATOR_BANCNOTE;
```

-Exemplificare waveform - NUMARATOR BANCNOTE:



11. Eliberare chitanță (CHITANTA_OP)

-Această entitate se ocupă cu eliberarea chitanței. Intrarea de “enable” este activă dacă suma introdusă de utilizator este una validă și bancomatul este disponibil să elibereze suma cu bancnotele pe care le are.

-Intrarea “optiune” provine de la un switch, pentru ‘0’ utilizatorul nu dorește eliberarea de chitanță iar pentru ‘1’, bancomatul va elibera chitanța. Optiunea este transmisă la variabila “iesire” care va fi afișată pe un led.

-Codul:

```
library IEEE;
use ieee.std_logic_1164.all;

entity CHITANTA is
(
    enable: in std_logic;
    CLK: in std_logic;
    optiune: in std_logic;
    iesire: out std_logic;
    finish: out std_logic);
end CHITANTA;
```

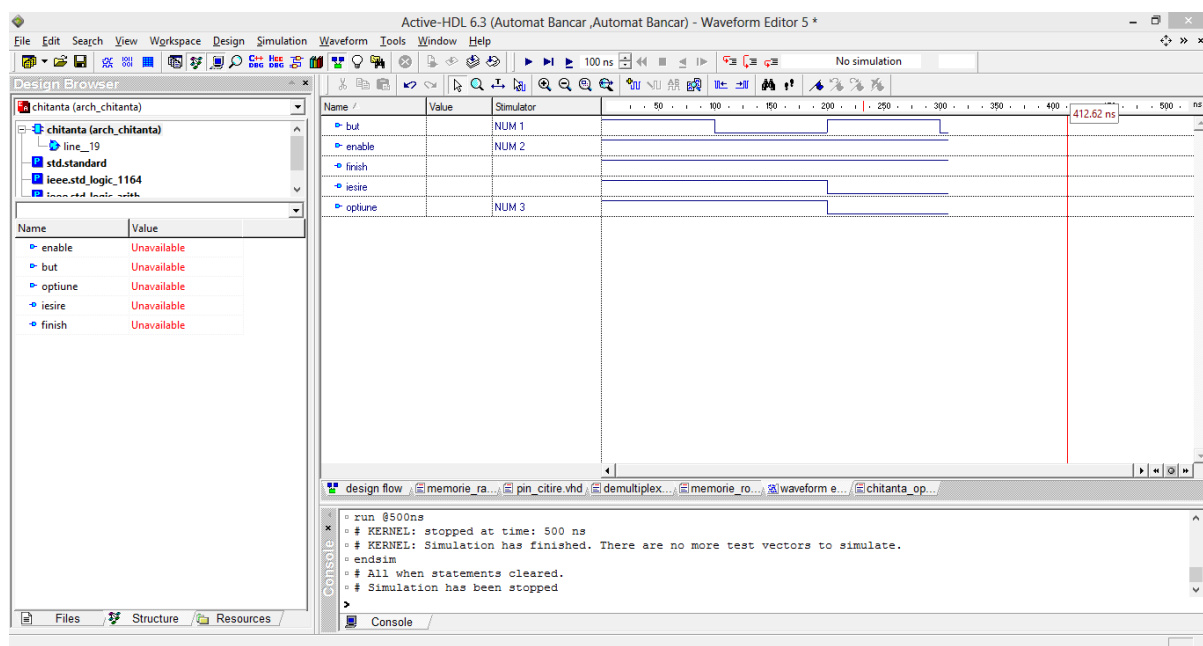
```

architecture ARCH_CHITANTA of CHITANTA is
begin
    process (CLK)
    begin
        if (CLK='1' and enable='1') then
            if optiune='0' then iesire<='0'; finish<='1';
            elsif CLK='1' then iesire<='1'; finish<='1'; --O -
fara chitanta , 1 -cu chitanta;
            end if;
        end if;
    end process ;

end ARCH_CHITANTA;

```

-Exemplificare waveform CHITANTA_OP:



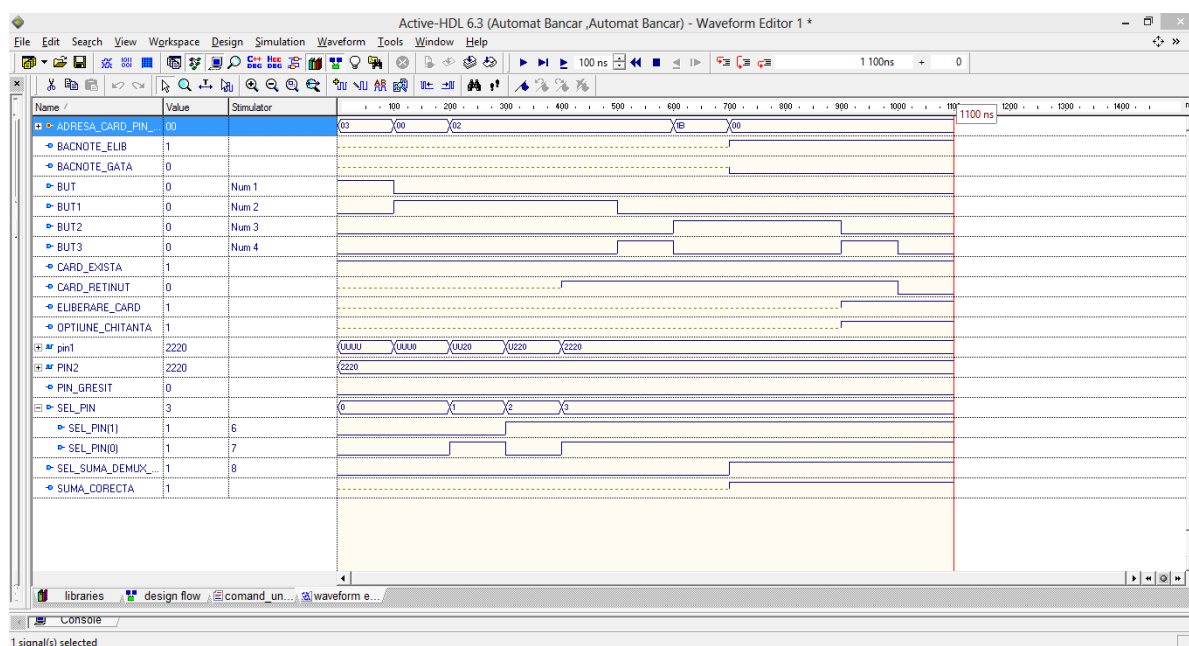
4. Justificarea soluției alese

Am optat pentru această soluție deoarece ni s-a parut că nu este foarte greu de implementat, și este relativ destul de aproape de realitate.

De asemenea, pot spune că soluția aleasă de noi este destul de ușor de înțeles în ciuda faptului că proiectul conține multe componente. Soluția este una eficientă pentru cerințele clientului.

Putem spune că această soluție este destul de apropiată de realitate, la fel în viața reală, un automat bancar citește de pe banda magnetică a cardului, numărul unic de înregistrare care coincide în cazul nostru cu vectorul de 5 biti "ADRESA_CARD" al automatului proiectat de către noi. După introducerea corectă a PIN-ului orice automat bancar așteaptă ca clientul să își aleagă operația următoare, având în cazul proiectului nostru, operațiile de retragere numerar și interogare sold. La sfârșitul operațiilor clientul poate opta pentru eliberarea chitanței pe care se afișează un scurt rezumat cu operațiile efectuate în sesiunea curentă.

Simulare ACTIVE-HDL proiect:



5. Instrucțiuni de utilizare

Acest automat pare destul de complicat, deoarece folosește mulți vectori mari, și multe semnale care leagă componentele între ele, însă dacă urmați pașii următori, nu va mai părea așa greu.

1. Se introduce în vectorul “ADRESA_CARD” echivalentul în binar a adresei cardului dumneavoastră, această adresă trebuie să fie cuprinsă între 0 și 31 în cazul automatului nostru.

2. Se acționează tactul automatului pentru ca prima componentă, “IDENTIFICATOR_CARD” să facă conversia adresei binare într-un întreg, care va reprezenta locul adresei din memoria ROM și RAM, unde sunt reținute codul PIN și respectiv soldul pentru contul respectiv. De asemenea se verifică validitatea contului introdus de utilizator.

3. Se introduce o secvență de 16 biți care reprezintă codul PIN introdus de utilizator, se acționează butonul de CLK pentru efectuarea comparării codului PIN cu cel citit din memoria ROM de la adresa respectivă, dacă cele două coincid se trece mai departe, altfel utilizatorul are posibilitatea să introducă de 3 ori codul PIN, după care în caz de eșec va fi reținut cardul, se aprinde LED-ul pentru “CARD REȚINUT”.

4. În cazul în care PIN-ul a fost introdus corect, pe intrarea S_DMUX se va introduce operația dorită (‘0’ pentru retragere numerar și ‘1’ pentru interogare sold).

A. Dacă alegem interogare sold, atunci se va acționa tactul general o singură dată, iar pe ieșirea SOLD_CLIENT1 va fi transmisă suma din memoria RAM corespunzătoare contului, aceasta fiind afișată la afișorul BCD7.

B. Dacă alegem retragere numerar, se va acționa tactul general care va activa componenta INTR_SUMA, după introducerea sumei pe 10 biți (tot câte 5 biți odată), se va verifica dacă suma este validă, mai mică decât 1000 și mai mică totodată decât soldul clientului din memoria RAM. În caz că suma introdusă este validă și bancomatul conține bancnotele necesare pentru eliberarea sumei cerute de client, se va efectua o actualizare a contului clientului, după retragerea sumei respective.

-Verificarea dacă bancomatul conține numărul necesar de bancnote se realizează prin intermediul componentei, NUMARATOR_BANCNOTE care se activează dacă suma introdusă este validă, acesta actualizează numărul de bancnote din interiorul bancomatului după operația efectuată de client.

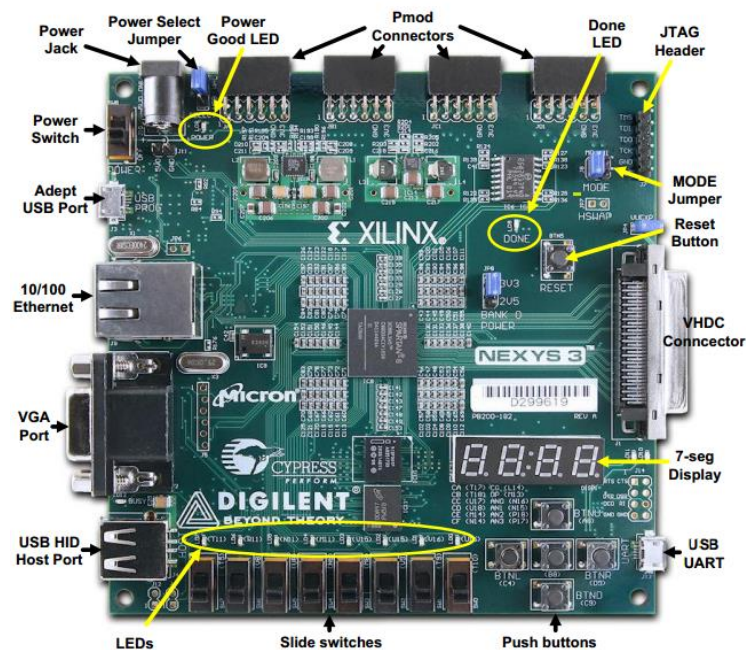
5. În funcție de operația aleasă de client, utilizatorul :

-În cazul retragerii de numerar, se ridică suma, cardul și eventual chitanța, în cazul în care s-a solicitat tipărirea acesteia.

-În cazul de interogare sold, clientul va solicita chitanța și se ridică cardul.

-Operația de eliberarea a chitanței se realizează prin componenta CHITANTA_OP, cu ajutorul unui switch, '0' dacă clientul nu dorește eliberarea chitanței și '1' în caz contrar.

Descrierea pinilor (Nexys3)



NET "adresa_card_pin_suma<0>" LOC = "T10";

NET "adresa_card_pin_suma<1>" LOC = "T9";

NET "adresa_card_pin_suma<2>" LOC = "V9";

NET "adresa_card_pin_suma<3>" LOC = "M8";

NET "adresa_card_pin_suma<4>" LOC = "N8";

NET "BUT" LOC = "B8";

NET "BUT1" LOC = "A8";
NET "BUT2" LOC = "C4";
NET "CLR" LOC = "D9";
NET "CLK_AFISOR" LOC="V10";
NET "SEL_PIN<0>" LOC="U8";
NET "SEL_PIN<1>" LOC="V8";
NET "SEL_SUMA_DEMUX_CHITANTA" LOC = "T5";

NET "a_g<0>" LOC = "T17";
NET "a_g<1>" LOC = "T18";
NET "a_g<2>" LOC = "U17";
NET "a_g<3>" LOC = "U18";
NET "a_g<4>" LOC = "M14";
NET "a_g<5>" LOC = "N14";
NET "a_g<6>" LOC = "L14";
NET "an<3>" LOC = "P17
NET "an<2>" LOC = "P18";
NET "an<1>" LOC = "N15";
NET "an<0>" LOC = "N16";
NET "CARD_EXISTA" LOC= "U16";
NET "ELIBERARE_CARD" LOC = "V16";
NET "CARD_RETINUT" LOC="U15";
NET "PIN_GRESIT" LOC= "V15";
NET "OPTIUNE_CHITANTA" LOC= "M11";
NET "BACNOTE_GATA" LOC = "N11";
NET "BACNOTE_ELIB" LOC= "R11";
NET "SUMA_CORECTA" LOC= "T11";

6.Posibilități de dezvoltare ulterioară

Versiunea concepută de către noi pentru automatul bancar, este una fără foarte multe opțiuni, conținând doar două dintre cele mai populare operații efectuate de aceste tipuri de automate, și anume interogare sold și retragere numerar. Desigur că ar mai putea fi adăugate câteva operații care apar la automatele moderne din zilele noastre, precum schimbarea PIN-ului , realizarea de transfer bancar între două conturi diferite sau crearea unor depozite cu suma aflată în contul clientului.

O altă deficiență a proiectului efectuat de către noi în comparație cu un automat bancar real, este legat de numărul limitat de conturi pe care îl putem stoca în memorie, doar 32 de conturi în cazul nostru, iar dacă ne raportăm la o bancă din zilele noastre, aceasta are milioane de clienți în baza de date, desigur această problema poate fi remediată prin mărirea capacității memoriilor, de exemplu dacă am realiza o extindere a memoriilor cu încă 15 biți , automatul nostru ar putea reține puțin peste un milion de conturi .

Încă un minus este reprezentat de faptul că bancomatul nostru poate să elibereze doar bancnote de 10 euro, fiind ușor de implementat astfel, însă dacă ne raportăm la un bancomat real, acesta este capabil de a elibera mai multe tipuri de bancnote, fiind generate de un algoritm inteligent de alegere a bancnotelor.